BSPD Testbench Project Specification Sheet

Authors: Chloe Gao, Trey Burnett
Team: Software
Reviewers: Chloe Gao, Trey Burnett

1. Introduction

1.1 Summary

This specification sheet outlines the technical scope and objectives for building a Hardware-in-the-Loop (HIL) testbench and safety firmware to validate the Brake System Plausibility Device (BSPD):

- **The HIL "Tester" Node:** A microcontroller setup that generates fake analog sensor signals (Throttle and Brake pressure) to simulate various track driving conditions and inject intentional hardware faults.
- **Automated Python Test Suite:** A script that automatically commands the Tester Node to run through edge-case scenarios and validates responses.

1.2 Context

The goal and vision for our embedded software subteam this year is to drastically improve our reliability and testing infrastructure before our code ever touches the physical car.

In Formula Student EV rules, the BSPD (Brake Sensor Plausibility Device) is a highly critical safety circuit. If the driver hits the brakes hard while the throttle is also pressed hard for more than 500 milliseconds, the car must immediately open the Shutdown Circuit and cut high-voltage power. In previous years, testing this logic involved manually probing wires, which is time-consuming and risks damaging expensive vehicle hardware.

We are heavily emphasizing automated safety testing this year. This project is an opportunity to get hands-on experience with embedded hardware, analog signals, and safety-critical state machines in a completely safe, low-voltage environment. By building a reliable Hardware-in-the-Loop (HIL) testbench, the team will gain a permanent tool to automatically verify our safety code against rule changes for years to come.

1.3 Goals and Technical Requirements

Goal 1: HIL Signal Generation (The "Tester" Node)

- **Functional Requirements:**
  - Output simulated analog voltages for Brake Pressure (0.5V - 4.5V) and Accelerator Pedal Position (APPS) (0.5V - 4.5V).
  - Change output voltages instantly or smoothly over time to simulate a driver pressing the pedals.
  - Receive and execute instructions from a connected computer (e.g., "Set APPS to 80%").
- **Technical Requirements:**
  - Built using a microcontroller (e.g., Teensy 4.0, ESP32, or Arduino) utilizing hardware Digital-to-Analog Converters (DACs) or RC-filtered PWM pins.
  - Establish a UART/Serial communication protocol to parse commands from a host PC.

Goal 2: Automated Python Test Suite

- **Functional Requirements:**
  - Automatically run through predefined driving scenarios without human intervention.
  - Test standard cases (e.g., normal driving), false positives (e.g., hard braking for only 200ms—should *not* trip), and hard faults (e.g., stuck throttle + panic braking—*must* trip).
  - Output a clear Pass/Fail terminal report at the end of the test cycle.
- **Technical Requirements:**
  - Write a Python script utilizing `pyserial` to communicate with the Tester Node.
  - Read back the digital trip state from the Target Node (either via a return serial message or a direct GPIO read) to verify if the VCU reacted in the correct amount of time.